

#21 · 斷路器面板

[優化] 技術代號 O-P1-3 · 約 3-4 天

一句話：看不到 AI 服務當機

為什麼要修

斷路器 (circuit breaker) 邏輯有，狀態散落在記憶體。

現在怎樣

server/jobs/circuitBreaker.ts 自動修復但無 UI；使用者反映「ElevenLabs 不能用」要等開發者去看 log。

問題在哪

故障診斷慢、運維反應差。

要改什麼

暴露 circuitBreakerRegistry 全域 Map + 新增 tRPC 暴露 + admin 頁加面板 + 強制重置按鈕。

涉及檔案

server/jobs/circuitBreaker.ts (暴露 registry Map)

server/routers/adminRouter.ts (system.circuitBreakers query + resetCircuitBreaker mutation)

client/src/pages/AdminPage.tsx (新增 system tab)

client/src/components/admin/CircuitBreakerPanel.tsx (新增)

Claude Code 提示詞 (複製貼上)

把下面這段話完整複製，貼到 Claude Code (或 /claude 對話)。Claude Code 會依此完成此項修復。完成後跑下方驗證步驟確認。

請依照計畫附錄 B.6 實作 O-P1-3 Circuit Breaker UI：

【第 1 部分：暴露 registry】

1. 修改 server/jobs/circuitBreaker.ts，新增全域：

```
```typescript
export const circuitBreakerRegistry = new Map<string, CircuitBreaker>();
export function getCircuitBreakerRegistry() { return circuitBreakerRegistry; }
```
```

確保所有 CircuitBreaker 建構時自動註冊到此 Map (key = breaker name)。

【第 2 部分：tRPC】

2. 在 server/routers/adminRouter.ts 新增：

```
```typescript
adminRouter
.query("system.circuitBreakers", async ({ ctx }) => {
 if (ctx.user?.role !== "admin") throw new TRPCError({ code: "UNAUTHORIZED" });
 const reg = getCircuitBreakerRegistry();
 return {
 breakers: Array.from(reg.values()).map((cb) => ({
 name: cb.name, state: cb.getState(),
 metadata: {
 failureThreshold: cb.failureThreshold,
 cooldownMs: cb.cooldownMs,
 consecutiveFailures: cb.consecutiveFailures,
 lastFailureTime: cb.lastFailureTime,
 secondsUntilProbe: cb.getState() === "OPEN" ? Math.max(0, Math.ceil((cb.lastFailureTime +
 cb.cooldownMs - Date.now()) / 1000)) : 0,
 },
 })),
 globalHealthScore: calculateHealthScore(reg),
 };
})
.mutation("system.resetCircuitBreaker", { input: z.object({ name: z.string() }), async resolve({ input, ctx }) {
 if (ctx.user?.role !== "admin") throw new TRPCError({ code: "UNAUTHORIZED" });
 const reg = getCircuitBreakerRegistry();
 const cb = reg.get(input.name);
 if (!cb) throw new TRPCError({ code: "NOT_FOUND" });
 cb.reset();
 reg.set(input.name, cb);
 return { name: input.name, state: cb.getState() };
}});
```
```

```
const cb = getCircuitBreakerRegistry().get(input.name);
if (!cb) throw new TRPCErrror({ code: "NOT_FOUND" });
cb.recordSuccess();
await appendAuditLog({ userId: ctx.user.id, action: "circuit_breaker_reset", resource: input.name });
});
---
```

3. 實作 `calculateHealthScore(registry)` — 例如所有 CLOSED 為 100、每個 HALF_OPEN -10、OPEN -30，clamp [0, 100]。

【第 3 部分：UI】

4. 新增 `client/src/components/admin/CircuitBreakerPanel.tsx`：

- 頂部 Health bar：globalHealthScore (>80 綠 / >50 黃 / 其他紅)
- Per-breaker card：name + state badge + countdown (OPEN 狀態顯示「X 秒後嘗試恢復」) + 連續失敗 + 強制重置按鈕 (OPEN 才出現)
- Alert section：若任何 breaker !== CLOSED，紅色警示橫幅

5. 整合到 AdminPage 「system」tab。

6. 自動 refresh：每 10 秒重新 query。

不要做的事：不要把 CB 邏輯改成「永久關閉」（保留 HALF_OPEN 自動探測）。不要對非 admin 暴露細節 (OAuth flow / 密鑰名稱)。不要做「手動觸發失敗」測試按鈕 (風險高)。

如何驗證做完了

Admin 頁開 system tab → Health bar 顯示 100/100 全綠。模擬一家 AI 服務當機 (mock API 5xx 多次) → 看到對應 breaker 變黃 → 達閾值變紅，countdown 開始倒數。按「強制重置」→ 變綠。

預估耗時

3-4 天